# EECS 3216 Final Project Report: Driver Monitoring System

**Names:** Artin Kiany, Owen Zhu
**Student numbers:** 219457969, 219756949
**Date:** Apr.07.2026

# Table Of Contents

**Abstract** ..... **2**

**Introduction & Background** ..... **3**

1.1 Controller Area Network (CAN) ..... 3

1.2 Packages/Libraries used ..... 3

1.2.1 SocketCAN ..... 4

1.2.2 arduino-mcp2515 ..... 4

1.3 Hardware ..... 4

1.3.1 CANable ..... 4

1.3.2 Motor driver ..... 5

**Explanation** ..... **5**

2.0 Overview ..... 5

2.1 Computer Vision ..... 5

**Results** ..... **6**

3.1 Challenges ..... 6

3.2 Next steps ..... 6

**Conclusion** ..... **7**

**References** ..... **8**

**Appendix** ..... **9**

**Keywords:** Driver Monitoring System (DMS), Advanced Driver Assistance Systems (ADAS), Controller Area Network (CAN), Computer Vision, Drowsiness Detection, Eye Aspect Ratio (EAR), Embedded Systems, Cyber-Physical Systems (CPS), Arduino, MCP2515, SocketCAN, Real-Time Systems

# Abstract

This project presents the design and implementation of a Driver Monitoring System (Driver Monitoring System) with a focus on driver monitoring within a cyber-physical framework. This system integrates computer vision software, which implements OpenCV hosted on a laptop to determine in real time whether a driver is drowsy, sending a signal to an Arduino microcontroller through a CAN (Controller Area Network) Bus, serving as a communication method to operate the signal between the CV and the car.

Upon detection of driver drowsiness, the system initiates a sequence of corrective actions designed to simulate real-world vehicle safety responses. These actions include repeatedly activating and deactivating DC motors to mimic braking and acceleration, as well as triggering an audible buzzer to alert the driver. This behavior continues until the system detects that the driver has returned to an alert state, at which point it transitions back to normal operation.

Communication using the CAN bus is widely used as a protocol in automotive systems which we seek to replicate. Using an Arduino microcontroller equipped with a MCP2515 CAN controller, the microcontroller will send signals to DC motors to simulate erratic vehicle movement by stopping and starting repeatedly.

Overall, this project seeks to show the integration of real-time computer vision, communication between embedded systems, and safety features found in modern vehicles, showing a practical application of CPS design principals.

# Introduction & Background

Modern vehicles increasingly rely on Advanced Driver Assistance Systems (ADAS) and Driver Monitoring Systems (DMS) to improve safety, particularly in addressing driver drowsiness, which is a leading cause of road accidents due to reduced reaction time and awareness. This project aims to design and implement a driver monitoring system within a cyber physical system (CPS) framework that detects drowsiness using computer vision and translates that into tangible system responses. By integrating an OpenCV-based eye detection algorithm with embedded systems and robust communication protocol (discussed below), the system translates software detection into physical responses such as motor control and alerts. The motivation behind this project is to explore how low cost, accessible hardware and open-source software can be combined to prototype a safety-critical system that is typically found only in advanced commercial vehicles. It is also inspired by features such as the Emergency Stop Assist developed by Volvo [1], which are designed to intervene when a driver becomes unresponsive. By researching and emulating similar concepts, this project aims to demonstrate how such safety mechanisms can be implemented on a smaller scale.

## 1.1 Controller Area Network (CAN)

CAN Bus is a communication protocol developed by Robert Bosch in 1983. It serves as the main communication system between all the different electronic control units (ECU) in modern cars. Unlike protocols like I2C or SPI, CAN operates without a single master or host controller. Instead, each ECU functions as an independent node within the network. These nodes are connected on a two wire bus consisting of CAN high (usually yellow) and CAN low (usually green). These wires are arranged as a twisted pair. This is because twisted pair cables reduce the electromagnetic interference and noise.

CAN operates as a peer-to-peer network in which any node can initiate communication. Before transmitting a frame, a node first checks whether the bus is busy or not, then simply puts the frame on the bus. CAN nodes don't have an address so communication is inherently broadcast-based. However, each frame has its own address known as the arbitration ID which determines its priority and meaning. Nodes use filters to read only the messages relevant to them, even though technically all nodes receive every frame on the bus. When multiple nodes transmit at the same time, CAN resolves the conflict through its built-in arbitration mechanism. Frames with lower arbitration IDs are given higher priority and allowed to proceed.

## 1.2 Packages/Libraries used

The following software libraries were used in this project:

- **Dlib** – for facial landmark detection and eye tracking
- **cv2 (OpenCV)** – for real-time image and video processing
- **NumPy** – for numerical computations and array handling

- **JSON** – for storing and loading calibration data
- **os** – for file and directory operations
- **scipy.spatial** – for calculating Euclidean distances (used in EAR computation)
- **time** – for handling delays and timing logic
- **can** – for CAN bus communication in Python (SocketCAN interface)

### 1.2.1 SocketCAN

SocketCAN is an open source package, which includes a set of CAN drivers for linux. It is a highly flexible and powerful tool used for debugging, monitoring, and logging CAN traffic. SocketCAN supports features such as filtering, raw frame access, and compatibility with various tools (e.g., candump (shows raw CAN frames on the bus) and cansend (transmits CAN frames)), making it especially useful for testing and analyzing CAN communication in real-time.

### 1.2.2 arduino-mcp2515

The arduino-mcp2515 library provides an interface for controlling the MCP2515 [Fig. 1] a stand-alone CAN controller. MCP2515 works as a CAN-to-SPI converter, enabling CAN communication on microcontrollers such as Arduino which don't have a dedicated CAN controller. The library includes functions for setting baud rates, and selecting operating modes (such as normal, loopback, and listen-only). Moreover, it supports message filtering, allowing nodes to process only relevant frames. The library also provides both polling and interrupt based methods for receiving data.

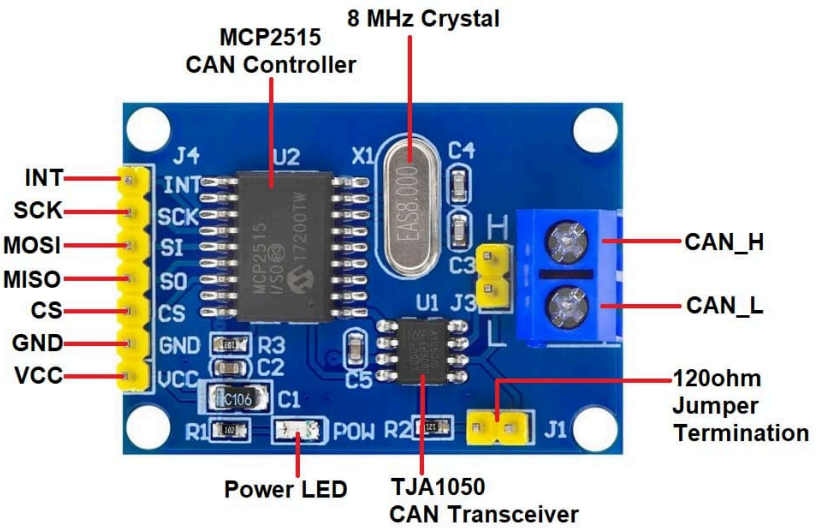


**Figure 1:** The MCP2515 CAN controller with SPI interface

## 1.3 Hardware

### 1.3.1 CANable

The CANable is an open-source USB-to-CAN adapter which uses an STM32 chip. To use this device we first flashed it with the candlelight firmware (which is again another

open-source project for USB-to-CAN adapters), then we connect to the host computer running linux. We bring up the CAN interface on the linux terminal using SocketCAN and set a baudrate of 500Kbps.

Our project consists of two CAN nodes, one being the car the other being the computer that hosted the CV model. The connection goes as follows: The arduino is connected to the MCP2515 through its SPI interface. Then the MCP2515 is connected to the CANable through its CANH and CANL pins. Finally the CANable is connected to the host computer.

### 1.3.2 Motor driver

The motor driver we used was the l298n which is a dual H-bridge motor driver module which is designed to control the speed and direction of DC motors. To control the speed it generates PWM signals

# Explanation

## 2.0 Overview

First a calibration script is used to gather 100 samples for both eyes closed and eyes opened to calibrate a personalised EAR value. After our computer vision model detects the user/driver's eyes. Using an algorithm it detects if the user has closed their eyes or not. If the driver has closed their eyes, the host computer sends a CAN signal with CAN ID 0X064, and a payload of 0x01 which indicates closed eyes (a payload of 0x00 is sent if the eyes are open). This message is then read by the MCP2515. The MCP2515 receives the CAN frame off the bus and stores it in one of its two internal receive buffers. It then pulls its INT pin (interrupt pin) low to signal the Arduino that a frame is waiting. This immediately triggers a hardware interrupt, pausing whatever the Arduino was doing before. It saves the current program counter and CPU registers onto the stack, and jumps to the Interrupt Service Routine canISR(). This sets an internal volatile interrupt flag. The CPU then restores its saved state and resumes exactly where it left off. The ISR is extremely short by design to prevent any missed interrupts from other sources. After that CAN frame is processed, and based on its payload it either continues moving the car forwards, or creates the jerking/jittering movement to wake the driver up. The jerking motion is implemented using a non-blocking timed FSM with two states stopped and forwards. It switches between each state every 300 ms.

## 2.1 Computer Vision

Computer vision is implemented via the OpenCV library which handles video/image processing. The library contains functions such as VideoCapture, cvtColor, and various draw functions to display text on-screen. OpenCV is used to capture video via webcam, convert to grayscale, then allow the second library: Dlib to detect EAR(Eye Aspect Ratio).

Dlib is a modern open-source C++ toolkit which contains pre-trained machine learning algorithms used to detect faces in the capture. face_detector relies on Dlib's get_frontal_face_detector which detects the presence of a human face, and a data file shape_predictor is used to draw 68 ordered landmarks on the face.

In the provided dat file, landmarks 36 to 48 provide the left and right eye landmarks, 3 landmarks for each eye are laid on the top line of the eye, and 3 more for each eye has landmarks on the bottom. To detect drowsiness, the euclidean distance is taken for these distances which will provide a value from 0-1; our EAR value.

By default, the EAR value will flag a warning once the value is below 0.2 and set a flag for drowsiness after 1 second, however, due to the nature of eyes, an EAR of 0.2 could be too high or too low of a threshold from person to person, this is why the calibration file calibrate.py exists. calibrate.py takes 100 samples for eyes open and 100 samples for eyes closed, writing each sample's EAR into a JSON. The main file finished.py will use the average of these 2 ratios, then use the value in between to determine the threshold, which will be the new EAR flag.

# Results

## 3.1 Challenges

CV-side, a challenge was that from person to person, the model would not reliably detect drowsiness due to poor EAR ratio thresholds. Response: Implemented and deployed calibrate.py to take individual EAR from person to person, creating a personally trained dataset that may be used in real-time for better drowsiness detection accuracy.

On the embedded systems side, a huge challenge had to do with interrupt handling and timing. The problem was that the actions that needed to be done when the interrupt occurred were all blocking. This is because it included using delays to jitter the car. However, it was really important to keep the Interrupt Service Routine (ISR) extremely short to avoid any missed interrupts or timing conflicts with other processes. This is why we went with the flag set architecture approach.

## 3.2 Next steps

On the hardware side our next steps are to enhance the system's response when driver drowsiness is detected by safely guiding the vehicle to the emergency lane. This will be done by adding two new sensors. One is an IR sensor to detect the lanes. The other are time of flight (TOF) sensors which measure the distance to nearby objects. The TOF sensors will be mounted on the side of the car to detect if there are any other vehicles in the surrounding lanes. If there are none, the car will keep changing lanes until it has arrived into the emergency lane and will fully stop there. Additionally, our goal is to use a raspberry pi running the QNX RTOS (real time operating system) as the main and only computer which will host the entire system including the CV model

Furthermore, currently the software side will only play a sound using pygame, and the signals sent to the CAN can still have a chance to fluctuate from eyes closed to eyes open. A better system would include having some sort of trigger detection for the transition between states, which will provide better accuracy.

# Conclusion

To conclude, the project was able to successfully integrate real-time computer vision into a cyber-physical systems framework that can be fine-tuned to each individual user. The system's ability to translate software-based detection into physical actions such as motor control and auditory alerts demonstrated that the project  was successful.

Hardware-wise, the embedded system also simulated a state machine, transitioning between drowsy and on, with guards between each transition.

While there are limitations, such as sensitivity to fluctuations in signals being sent and simplified vehicle simulation, the project provides a foundation for future improvements and overall showcases practical application of CPS principles.

# References


[1] “EX90 Emergency stop assist | Volvo Support EN-CA,” Sep. 08, 2025. https://www.volvocars.com/en-ca/support/car/ex90/article/47d2c97fd33effd3c0a8cc3718c999b7-e6113e73efebc320c0a8b0971179c732-8664b2fa77a7e089c0a8296870d1a409/

[2] *Controller Area Network (CAN) Protocol Overview*. (2006, August 31). NI. https://www.ni.com/en/shop/seamlessly-connect-to-third-party-devices-and-supervisory-system/controller-area-network--can--overview.html

[3] Davisking. (n.d.). *GitHub - davisking/dlib: A toolkit for making real world machine learning and data analysis applications in C++*. GitHub. https://github.com/davisking/dlib

[4] Electronics, C. (2025, January 23). CAN Bus Explained - A simple intro [2025]. *CSS Electronics*. https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial

[5] *How to use mcp2515 with jetson nano b01 4gb version by using a tutorial i already configured spi*. (2025, January 10). NVIDIA Developer Forums. https://forums.developer.nvidia.com/t/how-to-use-mcp2515-with-jetson-nano-b01-4gb-version-by-using-a-tutorial-i-already-configured-spi/319545

[6] Singh, V., & Singh, V. (2023, May 9). *Driver drowsiness detection using MediaPipe in Python*. LearnOpenCV – Learn OpenCV, PyTorch, Keras, Tensorflow With Code, & Tutorials. https://learnopencv.com/driver-drowsiness-detection-using-mediapipe-in-python

[7] *SocketCAN - Controller Area Network — The Linux Kernel documentation*. (n.d.). https://docs.kernel.org/networking/can.html

[8] Wikipedia contributors. (2026, April 2). *CAN bus*. Wikipedia. https://en.wikipedia.org/wiki/CAN_bus

# Appendix

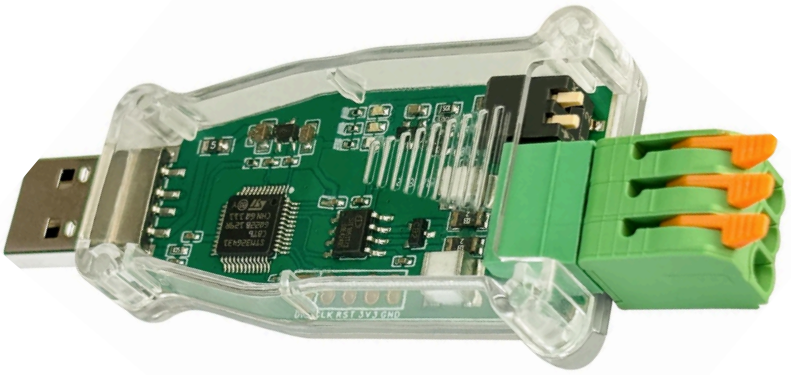

Figure 2: CANable USB-to-CAN converter

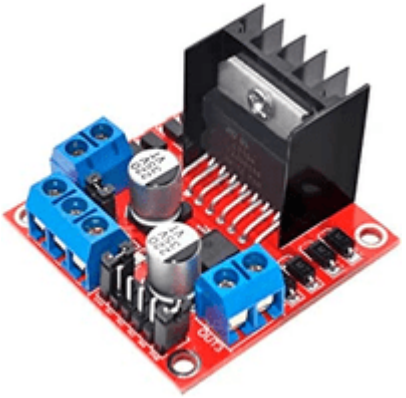

Figure 3: L298N motor driver module